

Berliner Hochschule für Technik

Bachelorarbeit

Konzeption und Implementierung einer interaktiven Visualisierung präemptiver Scheduling-Verfahren für die Lehre

Marten Kirschenhofer
Matrikelnummer: 833825
Studiengang: Medieninformatik

Betreuung: Prof. Dr. Christian Forler
Abgabedatum: 15.09.2026

Vorwort

Diese Bachelorarbeit entstand im Studiengang Medieninformatik an der Berliner Hochschule für Technik. Sie behandelt die Konzeption und Implementierung einer interaktiven Anwendung zur Veranschaulichung präemptiver CPU-Scheduling-Verfahren.

Ich widme diese Arbeit meinen lieben Schwiegereltern, die mir stets mit gutem Rat zur Seite standen, meiner wundervollen Verlobten, die mich auch in schwierigen Zeiten unterstützt hat, und meinem besten Freund, bei dem ich trotz des großen Stresses stets ich selbst sein konnte.

Contents

Vorwort	ii
List of Figures	v
List of Tables	vi
Abkürzungsverzeichnis	vii
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung der Arbeit	1
1.3 Thematische Ausrichtung der Arbeit	2
1.4 Vorgehensweise und Aufbau der Arbeit	2
1.5 Leitgedanke der Arbeit	3
2 Grundlagen	4
2.1 Betriebssysteme und Prozessverwaltung	4
2.2 CPU-Scheduling in Betriebssystemen	5
2.3 Präemptive versus nicht-präemptive Scheduling-Verfahren	5
2.4 Überblick über die betrachteten Algorithmen	7
2.5 Kriterien zur Bewertung von Scheduling-Verfahren	8
2.6 Stand der Forschung zu Scheduling-Visualisierungen	9
3 Anforderungsanalyse	10
3.1 Zielgruppe und Einsatzkontext	10
3.2 Fachliche Anforderungen	10
3.3 Didaktische Anforderungen	10
3.4 Interaktions- und Usability-Anforderungen	11
3.5 Technische Qualitätsziele	11
4 Systementwurf	12
4.1 Architekturprinzipien	12
4.2 Komponentenmodell der Anwendung	12
4.3 Interaktionsmodell für den Lernprozess	13
4.4 Human-Computer-Interaction als Qualitätsmerkmal	14
5 Implementierung	15
5.1 Technische Grundlagen und verwendete Technologien	15
5.2 Architektur der Anwendung	15
5.3 Datenmodell und Simulationskern	16
5.4 Umsetzung der Scheduling-Verfahren	16
5.5 Umsetzung der Visualisierung	17
5.6 Interaktionsmechanismen und Nutzerworkflow	17

5.7	Besondere Designentscheidungen	17
6	Qualitätssicherung	19
6.1	Qualitätsziele und Prüfstrategie	19
6.2	Automatisierte Verifikation der Simulationslogik	19
6.3	Accessibility und Bedienbarkeit	20
6.4	Reproduzierbarkeit und Vergleichbarkeit	20
6.5	Fazit zur Qualitätssicherung	20
6.6	Besondere technische Herausforderungen	21
6.7	Teststrategie und Qualitätssicherung	21
7	Ergebnis und Produktpräsentation	23
7.1	Entwickeltes Endprodukt	23
7.2	Produktüberblick und Nutzerfluss	23
7.3	Produktpräsentation mit Screenshots	24
7.4	Umgesetzte Funktionalität	24
7.5	Ergebnisbetrachtung	28
8	Fazit und Ausblick	29
8.1	Ergebnis der Arbeit	29
8.2	Ausblick	30
9	Anhang	32

List of Figures

4.1	Komponentenmodell der Anwendung mit den Ebenen Benutzeroberfläche, Anwendungslogik und Visualisierung. Die fachliche Simulation liefert die Daten, während die Darstellungsschicht die Ergebnisse verständlich aufbereitet.	13
4.2	Platzhalter für die Gesamtansicht des Scheduling-Visualizers.	14
7.1	Startansicht der Anwendung mit leerem Zustand und erklärender Ausgangslage. Der Nutzer startet mit der Definition eines Szenarios und der Auswahl eines Scheduling-Verfahrens.	24
7.2	Ladezustand eines konfigurierten Szenarios. Die Task-Daten sind sichtbar, und die Simulation kann mit einer klaren Ablaufsteuerung initialisiert werden.	25
7.3	Fokusansicht der Simulation mit Gantt-Zeitachse, aktiver Task-Darstellung und Kennzahlbereich. Diese Ansicht zeigt den Ablauf des gewählten Scheduling-Verfahrens in einer verständlichen, zeitlich strukturierten Form.	25
7.4	Detailansicht der Gantt-Darstellung. Einzelne Ausführungssegmente, Präemptionen und Task-Wechsel werden als konkrete Zeitabschnitte sichtbar gemacht.	26
7.5	Darstellung des Task-Status in der Warteschlangenansicht. Der aktuelle Zustand und die Warteliste werden gleichzeitig präsentiert, wodurch die Auswahlentscheidung des Algorithmus nachvollziehbar wird.	26
7.6	Kennzahlenbereich mit Warte-, Durchlauf- und Reaktionszeiten. Die Metriken geben eine kompakte fachliche Bewertung des Simulationslaufs wieder.	27
7.7	Vergleichsübersicht mit mehreren Läufen unter identischem Szenario. Die Darstellung ermöglicht den praktischen Vergleich verschiedener Scheduling-Verfahren anhand des selben Dateninputs.	27

List of Tables

Abkürzungsverzeichnis

CPU Central Processing Unit

HCI Human-Computer Interaction

MLFQ Multilevel Feedback Queue

RR Round Robin

SRTF Shortest Remaining Time First

1 Einleitung

1.1 Motivation und Problemstellung

Scheduling-Verfahren gehören zu den zentralen Konzepten moderner Betriebssysteme. Sie bestimmen, welcher Prozess zu welchem Zeitpunkt CPU-Zeit erhält und beeinflussen unmittelbar Reaktionsfähigkeit, Fairness und Auslastung eines Systems. Obwohl diese Verfahren in Lehrbüchern und Vorlesungen theoretisch sauber beschrieben werden, fällt es Studierenden häufig schwer, ihren zeitlichen Ablauf in konkreten Situationen zu verstehen [5, 7].

Gerade präemptive Verfahren sind für Lernende anspruchsvoll, weil sie viele Zustandswechsel, Unterbrechungen und Neuplanungen erzeugen. Die Entscheidungslogik bleibt bei reinen Textbeschreibungen und statischen Beispielen oft abstrakt. Das Problem zeigt sich besonders beim Vergleich verschiedener Verfahren unter identischen Eingaben: Die fachlichen Unterschiede sind zwar theoretisch bekannt, aber in ihrer unmittelbaren Wirkung auf Ablauf, Wartezeiten und Reaktionsverhalten nicht immer intuitiv nachvollziehbar.

Vor diesem Hintergrund wurde eine interaktive Webanwendung entwickelt, die Scheduling-Verfahren als sichtbaren, steuerbaren Ablauf modelliert. Ziel ist nicht die Beantwortung einer rein wissenschaftlichen Forschungsfrage, sondern die Erstellung eines nutzbaren, didaktischen Werkzeugs, das technische Zusammenhänge transparent macht und Studierenden einen praktischen Zugang zu den Grundlagen des CPU-Scheduling eröffnet.

1.2 Zielsetzung der Arbeit

Die vorliegende Arbeit verfolgt die Konzeption und Umsetzung einer interaktiven Webanwendung zur Veranschaulichung präemptiver Scheduling-Verfahren. Das Ziel besteht darin, eine Anwendung zu entwickeln, die:

- ausgewählte Scheduling-Verfahren fachlich korrekt simuliert,
- ihre zeitlichen Abläufe verständlich visualisiert,
- verschiedene Verfahren anhand identischer Szenarien vergleichbar macht,
- und als didaktisches Werkzeug im Lehrkontext nutzbar ist.

Der Fokus liegt damit auf der praktischen Umsetzung eines verständlichen und erweiterbaren

Produkts. Die Arbeit verbindet fachliche Grundlagen aus der Betriebssystemtheorie mit Prinzipien aus Software Engineering und Human-Computer Interaction. Dabei steht die nachvollziehbare Architektur, die klare Interaktion und die unmittelbare Produktwirkung im Vordergrund.

1.3 Thematische Ausrichtung der Arbeit

Die Arbeit ist bewusst nicht als klassische empirische Untersuchung konzipiert, sondern als engineering-orientierte Produktdokumentation. Der Schwerpunkt liegt auf der Gestaltung einer Anwendung, die im Studienkontext einen Mehrwert bietet und zugleich technische, didaktische und gestalterische Entscheidungen nachvollziehbar dokumentiert.

Diese Ausrichtung ist für den Studiengang geeignet, weil sie die Verbindung zwischen theoretischem Wissen und praktischer Umsetzung sichtbar macht. Die fachlichen Grundlagen aus der Betriebssystemlehre werden mit einer strukturierten Architektur, benutzerfreundlicher Oberfläche und konkreten Produktbeispielen verknüpft. Dadurch entsteht eine Bachelorarbeit, die nicht nur theoretisch fundiert ist, sondern auch die Entwicklung eines nutzbaren Softwaresystems als zentrale Leistung hervorhebt.

1.4 Vorgehensweise und Aufbau der Arbeit

Die Arbeit folgt einem engineering-orientierten Vorgehen:

1. Analyse der fachlichen Grundlagen zu CPU-Scheduling und relevanten Messgrößen,
2. Ableitung von Anforderungen an eine didaktische Visualisierungsanwendung,
3. Konzeption der Systemarchitektur und der Komponentenstruktur,
4. technische Umsetzung der Webanwendung,
5. produktorientierte Darstellung der Ergebnisse mit Screenshots und systematischer Erklärung.

Der Aufbau der Bachelorarbeit ist entsprechend auf die Entwicklung und Präsentation des Produkts ausgerichtet. Kapitel 2 behandelt die fachlichen Grundlagen. Kapitel 3 und Kapitel 4 legen die Anforderungen an die Architektur und die HCI-gestützte Gestaltung fest. Kapitel 5 beschreibt die konkrete Konzeption und den Entwurf der Anwendung. Kapitel 6 dokumentiert die technische Implementierung. Kapitel 7 stellt das Ergebnis mit der Produktpräsentation und der Visualisierung der wichtigsten Funktionen dar. Kapitel 8 schließt mit Reflexion, Grenzen und Ausblick.

1.5 Leitgedanke der Arbeit

Der zentrale Gedanke dieser Arbeit lautet:

Praktische Verständlichkeit entsteht nicht nur durch fachliche Korrektheit, sondern auch durch eine gut strukturierte Architektur, klare Interaktion und eine verständliche Darstellung des zeitlichen Ablaufs.

Diese Perspektive bildet den Kern der Arbeit. Sie macht deutlich, dass die Simulation von Scheduling-Verfahren nicht nur eine fachliche Aufgabe ist, sondern auch ein Design- und Architekturproblem mit unmittelbaren Auswirkungen auf Nutzbarkeit und Lernwirkung.

2 Grundlagen

Dieses Kapitel beschreibt die theoretischen Grundlagen, auf denen Konzeption, Implementierung und Evaluation der Anwendung aufbauen.

2.1 Betriebssysteme und Prozessverwaltung

Betriebssysteme koordinieren den Zugriff auf Hardware-Ressourcen und stellen Ausführungsumgebungen für Programme bereit. Eine Kernaufgabe ist dabei die Verwaltung konkurrierender Ausführungseinheiten, die um CPU-Zeit, Speicher und weitere Ressourcen konkurrieren [5, 7].

Historisch stand der Begriff des Prozesses im Mittelpunkt: Ein Prozess ist eine vollständige, isolierte Ausführungsumgebung mit eigenem Adressraum, Dateihandles und Ressourcenzustand. Prozesse durchlaufen typischerweise mehrere Zustände, etwa bereit, laufend und blockiert. Das Betriebssystem steuert Übergänge zwischen diesen Zuständen und bestimmt durch Scheduling-Entscheidungen, welcher bereite Prozess als nächstes ausgeführt wird. Diese Entscheidungen wirken sich direkt auf wahrgenommene Systemreaktion und Gesamtdurchlauf aus.

Parallel zur Prozessabstraktion existiert das Konzept des Threads (oder leichtgewichtigen Prozesses). Ein Thread ist eine Ausführungseinheit innerhalb eines Prozesses und teilt sich Speicher, offene Dateien und andere Ressourcen mit anderen Threads des gleichen Prozesses. Der Hauptunterschied liegt in der Isolation und dem Ressourcenaufwand: Ein Kontextwechsel zwischen Threads ist deutlich schneller als zwischen Prozessen, da weniger Systemzustand zurückgespeichert und wiederhergestellt werden muss. Threads erlauben es daher, eine feinere Parallelisierung innerhalb eines Programms zu erreichen.

Aus der Perspektive des CPU-Schedulers gibt es zwei Ebenen der Entscheidung: Auf Kernel-Ebene plant der Scheduler Prozesse oder Kernel-Level-Threads ein, während auf User-Level Scheduling-Strategien zusätzlich oder alternativ Anwendungs-Threads verwalten können. Moderne Systeme wie Linux arbeiten typischerweise mit einer Eins-zu-Eins-Abbildung (ein Kernel-Thread pro User-Thread), wodurch die Grenze zwischen Prozessen und Threads verschwimmt.

2.2 CPU-Scheduling in Betriebssystemen

CPU-Scheduling beschreibt die Auswahlstrategie für die Vergabe von Prozessorzeit an bereite Ausführungseinheiten – ob Prozesse, Kernel-Threads oder User-Level-Threads. Ziel ist ein sinnvoller Ausgleich zwischen konkurrierenden Kriterien wie kurzer Reaktionszeit, niedriger Wartezeit, hoher Auslastung und fairer Behandlung [4, 5].

In Mehrprogrammsystemen ist Scheduling eine kontinuierliche Entscheidung unter Unsicherheit, da Ankunftszeitpunkte und Verhalten der Ausführungseinheiten variieren. Deshalb sind Scheduling-Verfahren typischerweise als Heuristiken formuliert, deren Eignung vom Lastprofil und vom Systemziel abhängt. Obwohl Threads schnellere Kontextwechsel ermöglichen als Prozesse, unterliegen sie denselben Scheduling-Prinzipien: die Wahl des nächsten auszuführenden Elements, die Behandlung von Präemption und Kontextwechsel, und die Berechnung relevanter Metriken wie Wartezeit und Durchlaufzeit.

Aus didaktischer Perspektive konzentriert sich die vorliegende Anwendung auf die Prozess-Abstraktion, weil diese das Konzept klarer macht und die zugrunde liegenden Scheduling-Prinzipien ohne die zusätzliche Komplexität der Speicher- und Ressourcenteilung illustriert. Die dargestellten Verfahren sind aber unmittelbar auf Threads übertragbar und verdeutlichen daher auch die modernen Scheduling-Herausforderungen in Multi-Core- und Multi-Threading-Umgebungen.

2.3 Präemptive versus nicht-präemptive Scheduling-Verfahren

Eine grundlegende Klassifikation von Scheduling-Verfahren unterscheidet zwischen präemptiven und nicht-präemptiven Strategien. Dieser Unterschied hat tiefgreifende Auswirkungen auf die Verhalten von Systemen und auf die Komplexität von Analyse und Visualisierung.

2.3.1 Nicht-präemptive Verfahren

Nicht-präemptive Verfahren (auch kooperativ genannt) erlauben es einem Prozess, die CPU zu behalten, bis er selbst die Ausführung beendet oder sich aktiv freiwillig blockiert (etwa beim Warten auf eine Ressource oder Ein-/Ausgabe). Das Betriebssystem trifft Scheduling-Entscheidungen also nur an diskreten Punkten: beim Abschluss eines Prozesses oder beim Eintreten einer blockierenden Bedingung.

Vorteilhaft sind nicht-präemptive Verfahren, weil sie weniger Kontextwechsel verursachen und die Implementierung einfacher ist. Ein Prozess kann sich auf sein Verhalten konzentrieren, ohne plötzlich unterbrochen zu werden. Nachteilig ist jedoch, dass ein langläufiger oder fehlerhafter Prozess das ganze System blockieren kann und andere Prozesse unbegrenzt warten müssen. Nicht-präemptive Verfahren eignen sich daher eher für Batch-Systeme oder eingebettete Systeme mit vorhersagbarem Verhalten.

Historische Beispiele nicht-präemptiven Scheduling sind First Come First Served (FCFS) oder Shortest Job First (SJF), bei denen die Entscheidung jeweils beim Eintreffen eines neuen Prozesses oder beim Abschluss des aktuellen Prozesses getroffen wird.

2.3.2 Präemptive Verfahren

Präemptive Verfahren erlauben es dem Betriebssystem oder dem Scheduler aktiv, einen laufenden Prozess zu unterbrechen und einen anderen Prozess einzuplanen – unabhängig davon, ob der unterbrochene Prozess seine Arbeit abgeschlossen hat oder sich selbst freiwillig abgegeben hat. Eine Präemption wird typischerweise durch einen Timer-Interrupt (nach einem Zeitquantum) oder durch das Eintreffen eines höher priorisierten Prozesses ausgelöst.

Die Hauptvorteile präemptiven Scheduling liegen in der verbesserten Reaktionsfähigkeit und Steuerbarkeit. Kein einzelner Prozess kann das System monopolisieren, und interaktive Prozesse können schnell reagieren. Dies ist entscheidend für moderne, zeitkritische und interaktive Systeme wie Betriebssysteme für Desktops, Mobilgeräte oder Echtzeitsysteme. Die Kosten präemptiven Scheduling sind jedoch höher: Jede Präemption erfordert einen Kontextwechsel, bei dem Prozessorspeicher, Register und Zustand gespeichert und wiederhergestellt werden müssen. Dies verursacht Overhead und kann die Systemleistung reduzieren.

Ein zentraler Unterschied zwischen präemptiven und nicht-präemptiven Verfahren ist auch die zeitliche Struktur der Entscheidungen. Nicht-präemptive Verfahren treffen Entscheidungen nur beim Abschluss oder der Blockierung von Prozessen; präemptive Verfahren können Entscheidungen kontinuierlich oder in regelmäßigen Abständen treffen. Das macht präemptive Verfahren zeitlich komplexer und interessanter für Analyse und Visualisierung, da sich der Zustand des Systems häufiger ändert.

2.3.3 Herausforderungen präemptiver Verfahren

Die Visualisierung präemptiver Verfahren stellt besondere Anforderungen dar, da die Dynamik höher ist: Ein Prozess kann mehrmals unterbrochen werden, muss seinen Zustand zwischen Unterbrechungen speichern, und die Auswirkungen von Präemptionen auf Wartezeit, Durchlaufzeit und Kontextwechsel sind nicht unmittelbar offensichtlich. Genau deshalb ist interaktive Visualisierung hier besonders wertvoll – sie macht die sonst abstrakten Entscheidungen des Schedulers räumlich und zeitlich nachvollziehbar.

Die vorliegende Anwendung konzentriert sich ausschließlich auf präemptive Verfahren, weil diese die modernen Anforderungen widerspiegeln und gleichzeitig die pädagogische Herausforderung sind, die durch eine gute Visualisierung am besten gelöst wird.

2.4 Überblick über die betrachteten Algorithmen

Alle in der vorliegenden Anwendung implementierten Verfahren sind präemptiv. Sie repräsentieren verschiedene Strategien zur Auswahl des nächsten auszuführenden Prozesses und zur Handhabung von Unterbrechungen.

2.4.1 Last Come First Served (LCFS)

Last Come First Served (LCFS) priorisiert neu eingetroffene Prozesse. In der präemptiven Variante kann ein neu ankommender Prozess den aktuell laufenden Prozess sofort verdrängen. Das Verfahren ist anschaulich und verdeutlicht unmittelbar die Auswirkungen von Präemptionen: Ein lange laufender Prozess wird immer wieder unterbrochen, wenn neue Prozesse ankommen. Das Verfahren kann aber unter ungünstigen Lastprofilen zu langen Wartezeiten früher eingetrophener Prozesse führen und ist daher in der Praxis selten allein verwendet. Es dient jedoch didaktisch gut zur Illustration von Präemption.

2.4.2 Round Robin (Zeitscheiben-Verfahren)

Round Robin (RR) ist ein klassisches präemptives Verfahren, das jedem Prozess ein Zeitquantum (Time Slice) zuweist. Nach Ablauf des Quantums wird der Prozess durch den Scheduler unterbrochen (präemptiert) und wieder in die Bereitwarteschlange eingeordnet, während der nächste Prozess seine eigene Zeitscheibe erhält. RR gilt als faires Grundverfahren für zeitscheibenbasierte Systeme, da alle Prozesse zyklisch und regelmäßig CPU-Zeit erhalten. Die Präemption findet hier regelmässig nach festgelegten Zeitintervallen statt, was Round Robin didaktisch besonders interessant macht: Die Struktur der Unterbrechungen ist vorhersagbar und lässt sich leicht visualisieren [5].

2.4.3 Strict Priority (Prioritätsbasiertes Verfahren)

Strict Priority ist ein ereignisgesteuertes präemptives Verfahren, das stets den bereiten Prozess mit der höchsten Priorität auswählt. Trifft ein Prozess mit höherer Priorität ein während ein niedrig priorisierter Prozess läuft, verursacht dies sofort eine Präemption des laufenden Prozesses. Vorteilhaft ist die gezielte Bevorzugung kritischer Aufgaben und schnelle Reaktion auf wichtige Ereignisse; problematisch ist jedoch, dass niedrig priorisierte Prozesse lange warten müssen, wenn ständig höher priorisierte Prozesse ankommen. Strict Priority verdeutlicht daher den Zielkonflikt zwischen Responsivität und gleichmäßiger Ressourcennutzung [7].

2.4.4 Multilevel Feedback Queue (MLFQ)

Multilevel Feedback Queue (MLFQ) ist ein präemptives Verfahren, das mehrere Warteschlangenebenen mit unterschiedlichen Prioritäten und Zeitquanten kombiniert. Prozesse werden zunächst in die höchste Ebene eingeplant; wenn ein Prozess sein Zeitquantum auf dieser Ebene aufbraucht, wird er in eine niedrigere Ebene mit längerem Zeitquantum verschoben. Kurz laufende oder interaktive Prozesse profitieren davon, dass sie schnell auf der höchsten Ebene bearbeitet werden und die CPU schnell freigeben. CPU-intensive Prozesse werden schrittweise in niedrigere Ebenen verschoben, wo sie längere Zeitscheiben, aber niedrigere Priorität erhalten. Dadurch entsteht ein adaptives Verhalten, das auf heterogene Lastprofile mit gemischten Prozesstypen reagieren kann. MLFQ zeigt, wie Präemption kombiniert mit Feedback-Mechanismen ein ausgewogenes Verhalten ermöglichen kann [5, 2].

2.5 Kriterien zur Bewertung von Scheduling-Verfahren

Die Bewertung von Scheduling-Verfahren erfolgt über klar definierte Kennzahlen. Diese sind entscheidend, um Algorithmen nicht nur qualitativ, sondern auch quantitativ zu vergleichen [4].

2.5.1 Wartezeit

Die Wartezeit beschreibt die Zeitspanne, die ein Prozess im Zustand „bereit“ verbringt, ohne ausgeführt zu werden. Niedrige durchschnittliche Wartezeiten sind insbesondere in Systemen mit vielen kurzen Aufgaben wünschenswert.

2.5.2 Durchlaufzeit

Die Durchlaufzeit (Turnaround Time) ist die Zeit vom Eintreffen eines Prozesses bis zu dessen Abschluss. Sie bildet den Gesamtablauf aus Sicht eines Auftrags ab und ist eine zentrale Effizienzmetrik für batch-orientierte Lastprofile.

2.5.3 Reaktionszeit

Die Reaktionszeit beschreibt die Zeit vom Eintreffen eines Prozesses bis zu seiner ersten CPU-Zuteilung. Diese Kennzahl ist besonders relevant für interaktive Systeme, in denen frühes Feedback wichtiger sein kann als minimale Gesamtdurchlaufzeit [5, 7].

Ergänzend können Kontextwechselhäufigkeit, Präemptionsanzahl, CPU-Auslastung und Leerlaufanteile betrachtet werden, um Zielkonflikte zwischen Responsivität, Durchsatz und Ressourceneffizienz sichtbar zu machen.

2.6 Stand der Forschung zu Scheduling-Visualisierungen

Die Forschung zur Algorithmusvisualisierung zeigt, dass interaktive Darstellungen Lernprozesse deutlich unterstützen können, wenn Zustandswechsel, Warteschlangen und zeitliche Dynamiken gleichzeitig sichtbar gemacht werden. Besonders in technischen Fächern ist die Visualisierung von Abläufen häufig wirksamer als reine Textbeschreibungen, weil Lernende dadurch nicht nur das Ergebnis, sondern auch den Entstehungsprozess verstehen [1, 3, 6].

Für präemptive Scheduling-Verfahren ist diese didaktische Herausforderung besonders groß, da diese Verfahren durch häufige Kontextwechsel, unerwartete Unterbrechungen und Rückkehr in die Warteschlange eine dynamische, nicht-lineare Logik aufweisen. Die Auswirkungen von Präemptionen auf Reaktionszeit, Durchlaufzeit und Wartezeit sind durch statische Beispiele und Tabellen oft schwer nachvollziehbar. Genau deshalb sind interaktive Visualisierungen für das Verständnis präemptiver Scheduling-Verfahren besonders relevant: Sie erlauben es, die Entscheidungen des Schedulers in ihrer zeitlichen Reihenfolge zu beobachten, die Gründe für Präemptionen zu verstehen und die direkten Konsequenzen dieser Entscheidungen auf die Metriken zu sehen.

Die vorliegende Anwendung positioniert sich in diesem Bereich als didaktisches Werkzeug für präemptives Scheduling: Sie kombiniert fachlich korrekte Simulation mit einer verständlichen, zugänglichen Benutzeroberfläche und ermöglicht dadurch das Lernen durch direktes Experimentieren. Damit folgt sie dem in der Forschung beschriebenen Ansatz, dass Lernwirkung nicht nur durch fachliche Korrektheit, sondern auch durch klare Visualisierung und aktive Interaktion entsteht.

3 Anforderungsanalyse

Die fachlichen Anforderungen an die Anwendung ergeben sich aus der Kombination aus Betriebssystemtheorie, Didaktik und Software Engineering. Ziel ist nicht nur die fachlich korrekte Abbildung von Scheduling-Verfahren, sondern auch ihre verständliche, interaktive Darstellung für Lernende. Deshalb müssen die Kriterien für die Anwendung sowohl die technische Korrektheit der Simulation als auch die Verständlichkeit der Visualisierung berücksichtigen.

3.1 Zielgruppe und Einsatzkontext

Die primäre Zielgruppe sind Studierende in einführenden und vertiefenden Lehrveranstaltungen zu Betriebssystemen. Die Anwendung soll im Lernkontext helfen, zeitliche Entscheidungen bei präemptiven Verfahren nachvollziehbar zu machen. Der Einsatzbereich reicht von einer geführten Lehrveranstaltung bis hin zur selbstständigen Exploration einzelner Szenarien.

3.2 Fachliche Anforderungen

Aus fachlicher Sicht muss der Visualizer die wichtigsten Verfahren konsistent modellieren: Round Robin, LCFS, Strict Priority und MLFQ. Die Anwendung muss dabei die relevanten Parameter berücksichtigen, insbesondere Ankunftszeit, Burst-Zeit, Priorität, Zeitquantum und Präemption. Zusätzlich müssen die wichtigsten Kennzahlen zusammen mit dem Verlauf sichtbar sein: Wartezeit, Durchlaufzeit, Reaktionszeit, Kontextwechsel und Präemptionsanzahl.

Die zentrale Anforderung lautet deshalb: Simulationslogik, Ereignisprotokoll und Kennzahlen müssen aus derselben Zustandsbasis abgeleitet werden. Dadurch bleiben die dargestellten Ergebnisse konsistent und nachvollziehbar.

3.3 Didaktische Anforderungen

Die didaktische Qualität der Anwendung ist für den Projekterfolg zentral. Lernende sollen nicht nur das Ergebnis sehen, sondern auch erkennen, warum ein Prozess zu einem bestimmten Zeitpunkt ausgewählt, unterbrochen oder in eine Warteschlange verschoben wird.

Deshalb müssen Zustandswechsel sichtbar und erklärbar sein. Ein gutes Lehrwerkzeug muss nicht nur korrekt, sondern auch verständlich und steuerbar sein.

Drei Leitprinzipien sind dabei besonders wichtig:

- **Transparenz:** Start, Präemption, Kontextwechsel und Abschluss müssen nachvollziehbar dargestellt werden.
- **Vergleichbarkeit:** Mehrere Runs unter identischem Szenario sollen direkt gegenübergestellt werden können.
- **Kontrollierbares Lerntempo:** Schrittweises und automatisches Abspielen müssen beide möglich sein.

3.4 Interaktions- und Usability-Anforderungen

Die Oberfläche soll die kognitive Last der Nutzenden reduzieren. Die wichtigsten Aufgaben müssen ohne unnötige Hürden durchgeführt werden können: Szenario definieren, Algorithmus auswählen, Ablauf starten und Ergebnis interpretieren. Die UI soll daher klar strukturiert sein und die Funktionen in sinnvoller Reihenfolge bereitstellen.

Für die Lernanwendung gelten zusätzlich Anforderungen an Verständlichkeit und Zugänglichkeit. Die wichtigsten Statusinformationen müssen direkt sichtbar sein, und die Interaktion muss mit wenigen Schritten und ohne Mehrdeutigkeit erfolgen. Die Benutzungsoberfläche soll außerdem als didaktisches Hilfsmittel und nicht als technische Black Box wirken.

3.5 Technische Qualitätsziele

Neben fachlicher Korrektheit und didaktischer Verständlichkeit ist die technische Qualität entscheidend. Die Anwendung soll modular, deterministisch und erweiterbar sein. Eine klare Trennung zwischen Simulationslogik, Zustandsverwaltung und Visualisierung reduziert Fehleranfälligkeit und erleichtert spätere Erweiterungen, etwa neue Algorithmen, neue Kennzahlen oder alternative Darstellungen.

Diese Qualitätsziele sind nicht nur technische Feinheiten, sondern tragen direkt zur Produktqualität und zur Nachvollziehbarkeit bei. Sie machen die Anwendung zu einem verständlichen, wiederverwendbaren Softwaresystem und nicht nur zu einer demonstrativen Demoszene.

4 Systementwurf

Die Anwendung basiert auf der Idee, dass fachliche Korrektheit und nachvollziehbare Interaktion nicht getrennt betrachtet werden sollten. Ein Scheduling-Visualizer muss nicht nur Ergebnisse berechnen, sondern auch erklären, warum bestimmte Prozesse zu einem Zeitpunkt laufen und andere warten. Deshalb ist die Architektur der Anwendung direkt mit den Anforderungen an die Interaktion verbunden.

4.1 Architekturprinzipien

Die Anwendung folgt einer komponentenbasierten Struktur mit klaren Verantwortlichkeiten. Die wichtigsten Prinzipien sind eine strikte Trennung zwischen fachlicher Simulationslogik und Präsentation, die deterministische Verarbeitung identischer Eingaben und die Modularisierung der wichtigsten Funktionen. Dadurch bleibt der Simulationskern testbar und die UI kann als Vermittler zwischen Fachlogik und Nutzer wirken.

Diese Struktur folgt im Kern gängigen Software-Engineering-Prinzipien. Die Kopplung zwischen den Bereichen bleibt niedrig, die Zuständigkeiten sind klar und neue Funktionen wie zusätzliche Algorithmen oder Visualisierungsansichten können ergänzend eingebaut werden, ohne die gesamte Architektur zu verändern.

4.2 Komponentenmodell der Anwendung

Das System lässt sich in drei Ebenen aufteilen:

- **Szenario- und Konfigurationslogik:** Verwaltung der Taskdaten und der Eingabeparameter.
- **Simulationskern:** Berechnung des Ablaufes anhand des gewählten Algorithmus.
- **Darstellung und Steuerung:** Gantt-Zeitachse, Stack-Visualisierung, Kennzahlen und Vergleichsübersicht sowie Steuerungselemente (Start, Pause, Schrittmodus, Zeitnavigation).

Die Steuerungslogik ist dabei integral in die Darstellungsschicht eingebunden. Sie ist nicht eine separate Verarbeitungsebene, sondern ein Bestandteil der Benutzeroberfläche, die den Simulationsablauf steuert und visualisiert. Diese Trennung ist für die didaktische Qualität der Anwendung entscheidend. Der Nutzer sieht nicht nur ein fertiges Ergebnis, sondern

den Ablauf in seiner zeitlichen Abfolge, die er durch Steuerungselemente selbst veranlasst. Die fachliche Entscheidung und ihre Präsentation bleiben in derselben Logik verknüpft, aber nicht in derselben Schicht zusammengefasst.

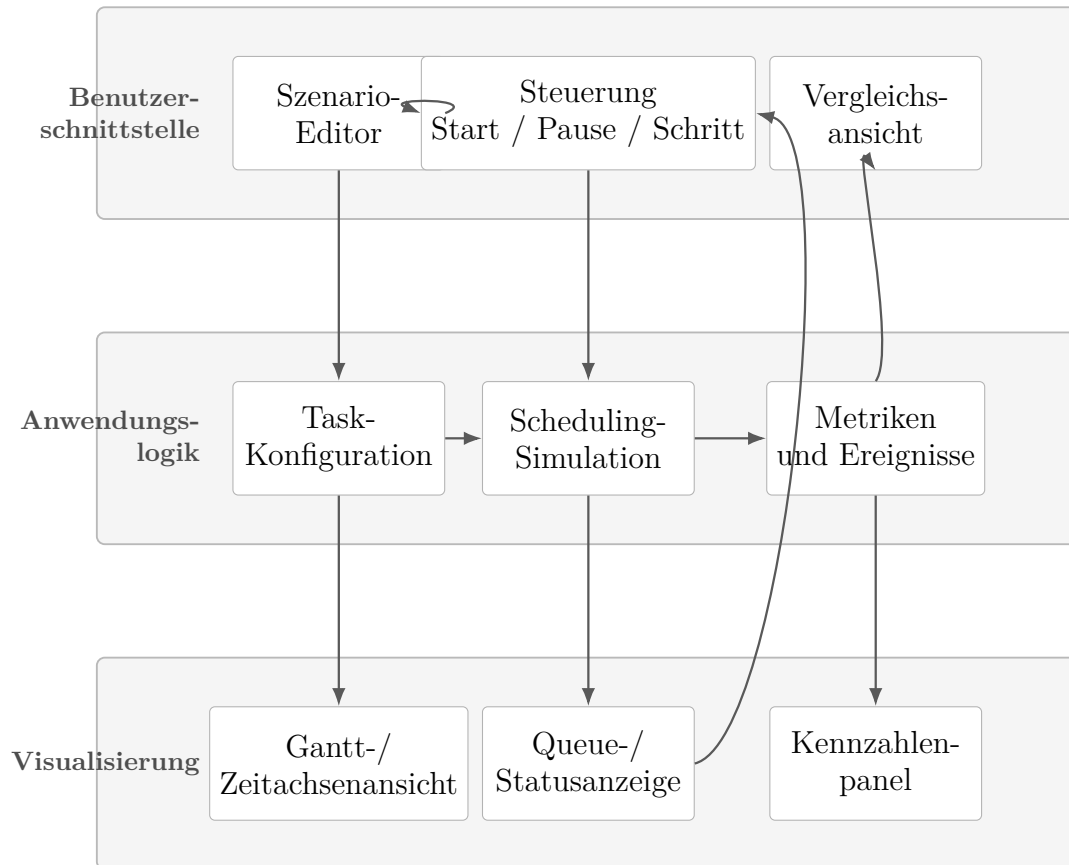


Figure 4.1: Komponentenmodell der Anwendung mit den Ebenen Benutzeroberfläche, Anwendungslogik und Visualisierung. Die fachliche Simulation liefert die Daten, während die Darstellungsschicht die Ergebnisse verständlich aufbereitet.

Die Pfeile in der Abbildung stellen den Datenfluss dar: Eingaben fließen von der Benutzeroberfläche durch die Anwendungslogik zur Visualisierung. Die Steuerungselemente (Start, Pause, Schritt) bewirken Neuberechnungen in der Simulationslogik, deren Ergebnisse dann sofort in den Visualisierungskomponenten dargestellt werden.

4.3 Interaktionsmodell für den Lernprozess

Die Interaktion der Anwendung folgt einem klaren Nutzungsfluss: Erst wird ein Szenario definiert, dann ein Algorithmus ausgewählt, anschließend wird der Ablauf gesteuert und schließlich vergleichend analysiert. Dieser Ablauf entspricht der kognitiven Logik der Lernenden: Die Bedienung ist nicht als bloße Technik, sondern als Hilfsmittel zur fachlichen Erkenntnis gestaltet.

Die wichtigsten interaktiven Elemente sind deshalb nicht allein „Buttons“, sondern Lern-

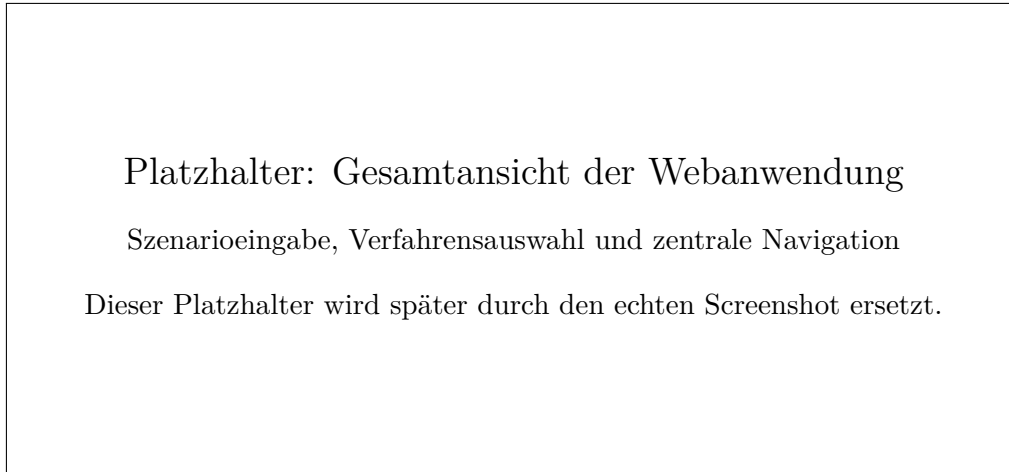


Figure 4.2: Platzhalter für die Gesamtansicht des Scheduling-Visualizers.

werkzeuge. Der Zeitstrahl zeigt, wer wann läuft; die Stack-Visualisierung erklärt, welche Prozesse in der Warteschlange warten; die Kennzahlen bilden die fachliche Zusammenfassung der Laufzeitdynamik. Dieses Zusammenspiel ist das eigentliche Interaktionsmodell der Anwendung.

4.4 Human-Computer-Interaction als Qualitätsmerkmal

Für eine Lernanwendung ist die Benutzeroberfläche kein Nebenaspekt, sondern ein integraler Bestandteil der fachlichen Aussage. Gute HCI in diesem Kontext bedeutet, dass:

- relevante Zustände direkt sichtbar sind,
- wichtige Ereignisse klar gekennzeichnet sind,
- Bedienung und fachlicher Inhalt in einem konsistenten Ablauf zusammenwirken,
- und die Visualisierung den Nutzer an der richtigen Stelle in den Lernprozess führt.

Dadurch wird die Anwendung nicht nur nutzbar, sondern auch lernförderlich. Sie macht die fachlichen Zusammenhänge verständlich, ohne die formale Korrektheit der Simulation zu verlieren. Genau diese Verbindung aus Technik, Didaktik und sichtbarer Interaktion ist der Kern des Projekts.

5 Implementierung

Die technische Umsetzung der Webanwendung orientiert sich direkt an den im vorangehenden Kapitel definierten fachlichen und didaktischen Anforderungen. Ziel dieses Kapitels ist die konkrete Realisierung eines interaktiven Scheduling-Visualizers, der fachlich korrekte Simulationsergebnisse mit einer verständlichen und steuerbaren Oberfläche verbindet. Dabei bleibt die fachliche Logik von der Präsentation getrennt, sodass die Visualisierung immer auf nachvollziehbare Berechnungsergebnisse zurückgeführt werden kann.

5.1 Technische Grundlagen und verwendete Technologien

Die Anwendung wurde als clientseitige Webanwendung mit Vue 3 und TypeScript umgesetzt. Vue 3 dient als Basis für die komponentenbasierte UI-Struktur, während TypeScript die Modellierung von Tasks, Events, Snapshots, Runs und Kennzahlen typensicher macht. Vite übernimmt den Build- und Entwicklungsprozess, Vitest dient der automatisierten Verifikation der Kernlogik und GSAP ergänzt die Oberfläche mit gezielten Übergängen und einer klaren dynamischen Präsentation ohne die fachliche Logik der Simulation zu beeinflussen.

Die zentrale technische Entscheidung ist die Trennung von fachlicher Modellierung, Laufsteuerung und Visualisierung. Dadurch entsteht ein klarer Datenfluss: Eingabedaten werden in eine deterministische Simulation überführt, die Ergebnisse werden als Ereignisliste, Segmente und Kennzahlen modelliert und anschließend in der Oberfläche als Gantt-Darstellung, Queue-Ansicht und Vergleichsübersicht visualisiert.

5.2 Architektur der Anwendung

Die Architektur folgt dem in Kapitel 4 beschriebenen Schichtenmodell mit drei Ebenen:

- **Szenario- und Konfigurationslogik:** Verwaltung von Taskdaten, Parametern, Szenario-Definitionen und gespeicherten Laufkonfigurationen.
- **Simulationskern:** Deterministische Berechnung der Scheduling-Strategien auf Basis eines gemeinsamen Zustandsmodells.
- **Darstellung und Steuerung:** Gantt-Darstellung, Warteschlangenvisualisierung,

Kennzahlbereiche und Interaktionssteuerung für Abspielen, Pausieren und Vergleichen.

Diese Gliederung ist nicht nur aus Sicht des Software-Engineering sinnvoll, sondern entspricht auch den didaktischen Anforderungen der Anwendung. Der Nutzer kann den fachlichen Ablauf nachvollziehen, ohne die technische Struktur der Oberfläche direkt mit der Berechnung vermischen zu müssen.

5.3 Datenmodell und Simulationskern

Die fachliche Grundlage bildet ein gemeinsames Task-Modell mit Attributen wie Ankunftszeit, Burst-Zeit, Priorität, Verlauf, Restzeit und Status. Auf dieser Grundlage werden die einzelnen Scheduling-Verfahren als Varianten eines gemeinsamen Ablaufmodells umgesetzt. Dadurch bleiben die Verfahren unter identischen Bedingungen vergleichbar, und die Unterschiede im Ergebnis sind fachlich auf die jeweilige Algorithmik zurückführbar.

Der Kern erzeugt nicht nur ein Endergebnis, sondern mehrere, logisch zusammengehörige Datensätze:

- **Ereignislog:** chronologische Information über Dispatches, Präemptionen, Kontextwechsel und Abschluss.
- **Segmente:** zeitlich abgegrenzte Ausführungsschritte für die Gantt-Darstellung.
- **Snapshots:** Zustandsbilder für die Wiedergabe und die genaue Zeitmarkierung im Ablauf.
- **Metriken:** aggregierte Kennzahlen für Wartezeit, Durchlaufzeit, Reaktionszeit und Kontextwechsel.

Diese Struktur ist besonders wichtig für die fachliche Nachvollziehbarkeit: Die Visualisierung bezieht sich immer auf dieselben Daten, die auch der kennzahlenbasierte Vergleich verwendet. So bleiben die Darstellung und die Bewertung konsistent.

5.4 Umsetzung der Scheduling-Verfahren

Die Simulation modelliert das Verhalten der wichtigsten präemptiven Verfahren innerhalb eines identischen Simulationsrahmens. Round Robin wird durch Zeitquantum-Logik realisiert, Strict Priority durch prioritätsbasierte Auswahl mit Präemptionsentscheidungen, LCFS über LIFO-basierte Warteschlangenlogik und MLFQ über mehrere Ebenen mit reduzierter Priorität bei längeren Ausführungsintervallen.

Die zentrale Gemeinsamkeit aller Verfahren liegt in der konsistenten Tick- und Ereignislogik. Jeder Lauf wird auf Grundlage desselben Szenarios und derselben Zustandsbasis berechnet, sodass Unterschiede in Laufzeit, Wartezeit und Durchlaufzeit tatsächlich auf die

Strategiedefinitionen zurückzuführen sind und nicht auf abweichende Berechnungsmethoden.

5.5 Umsetzung der Visualisierung

Die Oberfläche ist in mehrere, logische Bereiche aufgeteilt: Szenario- und Parameterwahl, Laufsteuerung, Fokusansicht, Vergleichsübersicht und Kennzahlenbereich. Die Gantt-Zeitachse bildet den Ablauf eines einzelnen Laufes ab, die Stack-Visualisierung zeigt den aktuellen Zustand der Warteschlange und die Kennzahlen verknüpfen den Lauf mit relevanten fachlichen Metriken.

Die Anwendung nutzt SVG-basierte Darstellungen, um Zeitabschnitte, Segmentgrenzen, Ankunftsereignisse und Präemptionen präzise in der Oberfläche zu markieren. Dadurch entsteht eine visuell klare, fachlich genaue Darstellung, die zugleich als Lehrmittel genutzt werden kann. Die Visualisierung ist daher nicht nur dekorativ, sondern trägt unmittelbar zum Verständnis von Scheduling-Entscheidungen bei.

5.6 Interaktionsmechanismen und Nutzerworkflow

Die Interaktion folgt einem didaktisch klaren Ablauf: Szenario definieren, Algorithmus wählen, Lauf steuern, Ergebnis interpretieren und vergleichen. Die Wiedergabelogik wird getrennt von der Simulation modelliert. Sie liest vorberechnete Snapshots und zeigt nur den jeweils aktuellen Ablaufzustand an, ohne den fachlichen Berechnungsprozess erneut zu starten.

Wichtige Interaktionsfunktionen sind Start, Pause, Schrittmodus, Geschwindigkeitsanpassung und Zeitsprung. Ergänzend gibt es einen Vergleichsmodus, der mehrere Läufe auf identischer Eingabebasis nebeneinander darstellt. Dadurch wird die fachliche Bedeutung der Algorithmuswahl unmittelbar erfahrbar und mit den entsprechenden Kennzahlen verknüpft.

5.7 Besondere Designentscheidungen

Eine zentrale Designentscheidung war die Verbindung von fachlicher Korrektheit und didaktischer Transparenz. Die Anwendung zeigt nicht nur das Ergebnis, sondern auch die Zustandswechsel, die zu einer Entscheidung geführt haben. Dadurch wird aus einer bloßen Simulation ein erklärbares Lehrwerkzeug.

Ein weiterer wichtiger Punkt ist die klare Trennung zwischen Berechnung und Vergleich. Die Laufberechnung erzeugt die Basisdaten, während die Vergleichsansicht diese Daten auswertet und visuell gegenüberstellt. Diese Struktur verbessert die Wiederverwend-

barkeit, vereinfacht die Testbarkeit und erleichtert zukünftige Erweiterungen um weitere Algorithmen oder Lernpfade.

6 Qualitätssicherung

Mit der Umsetzung der Kernfunktionalität endet die reine Entwicklung nicht automatisch. Für ein didaktisches Werkzeug mit fachlicher Relevanz ist die Qualität der Simulation, der Interaktion und der Wiederholbarkeit ebenso wichtig wie die technische Funktionsfähigkeit selbst. Dieses Kapitel beschreibt deshalb die Maßnahmen zur Sicherung von Korrektheit, Bedienbarkeit und nachvollziehbarer Verifikation der Anwendung.

6.1 Qualitätsziele und Prüfstrategie

Die Qualitätsstrategie verfolgt drei Ziele: fachliche Korrektheit der Simulationslogik, robuste Bedienbarkeit der Oberfläche und reproduzierbare Ergebnisse bei identischer Eingabe. Diese Ziele sind besonders wichtig, weil die Anwendung nicht nur ein Produkt demonstriert, sondern auch als Lern- und Analyseinstrument eingesetzt werden soll. Ein Fehler in der Selektionslogik oder ein inkonsistenter Zustand in der Wiedergabe wäre für didaktische Vermittlung unmittelbar problematisch.

Aus diesem Grund wurde ein mehrstufiger Prüfansatz gewählt. Die Simulation wird auf fachliche Korrektheit getestet, die Oberfläche auf Interaktions- und Zugänglichkeitsaspekte überprüft und die Datenflüsse zwischen Szenario, Lauf und Kennzahlen auf konsistente Zustandsübergänge geprüft. Die jeweiligen Prüfmaßnahmen sind damit nicht bloß technische Nebenarbeiten, sondern Teil der fachlichen Verlässlichkeit der Anwendung.

6.2 Automatisierte Verifikation der Simulationslogik

Die Kernlogik der Anwendung wird mit den in der Projektstruktur vorhandenen Vitest-Tests überprüft. Dabei werden insbesondere die deterministischen Berechnungsschritte der Scheduling-Verfahren, die Behandlung von Zeitquanten und Präemptionen sowie die korrekte Berechnung von Lauf- und Wartezeiten validiert. Die Tests sind bewusst so angelegt, dass sie die einzelnen Algorithmik-Entscheidungen nicht endlos durch UI-Komponenten replizieren, sondern die fachliche Logik direkt auf modellierter Datenbasis prüfen.

Diese Prüfstrategie ist für die didaktische Verwendung kritisch, weil sie sicherstellt, dass die in der Gantt-Darstellung gezeigten Zustände auch im fachlichen Sinne korrekt sind. Wenn ein Prozess aufgrund eines Zeitquantums unterbrochen oder nach einer Prioritätsänderung weitergeführt wird, muss die Logik dieselbe Entscheidung auch in den Kennzahlen und im

Ereignislog widerspiegeln. Genau diese Konsistenz ist durch die automatisierten Tests abgesichert.

6.3 Accessibility und Bedienbarkeit

Neben der fachlichen Korrektheit wurde die Bedienbarkeit der Oberfläche systematisch berücksichtigt. Die Projektstruktur enthält Tests für Barrierefreiheit und Interaktionslogik, die insbesondere die semantische Struktur, fokussierbare Komponenten und klare Bedienbarkeit prüfen. Die Anwendung wird damit nicht nur unter funktionalen Gesichtspunkten bewertet, sondern auch hinsichtlich ihrer Zugänglichkeit und Nutzerführung.

Ein Lernwerkzeug ist nur dann wirksam, wenn die zentrale Information akustisch und visuell verständlich vermittelt wird und Nutzerinnen und Nutzer die relevanten Steuerungselemente ohne Verwirrung bedienen können. Für diesen Zweck sind saubere Statusmeldungen, verständliche Fokusführung und konsistente Bedienkonventionen von besonderer Bedeutung.

6.4 Reproduzierbarkeit und Vergleichbarkeit

Ein weiteres Qualitätsmerkmal der Anwendung ist die Reproduzierbarkeit von Läufen. Bei identischer Eingabe und identischen Parametern muss derselbe Ablauf erzeugt werden. Diese Eigenschaft ist für Lernumgebungen besonders wichtig, da sie die Vergleichbarkeit zwischen verschiedenen Scheduling-Verfahren unterstützt. Dadurch können Prozesse, Wartezeiten und Kontextwechsel in derselben Ausgangsbasis miteinander verglichen werden, ohne dass zusätzliche, nicht nachvollziehbare Einflussfaktoren die Interpretation verfälschen.

Darüber hinaus ist die Vergleichsübersicht als Qualitätsmerkmal der Anwendung zu verstehen: Sie erlaubt die direkte Gegenüberstellung mehrerer Runs und zeigt an, an welchen Stellen sich die Verfahren systematisch unterscheiden. Die Bewertungslogik ergänzt diese Analyse um eine wertungsbasierte Perspektive, ohne die fachliche Korrektheit der Simulation zu überschreiben.

6.5 Fazit zur Qualitätssicherung

Die Qualitätssicherung der Anwendung ist daher nicht als nachträgliche Kontrolle, sondern als integraler Bestandteil der Umsetzung zu verstehen. Fachliche Korrektheit, wiederholbare Simulationsergebnisse und verständliche Benutzerführung bilden zusammen die Grundlage dafür, dass die Anwendung im didaktischen Kontext belastbar und zuverlässig eingesetzt werden kann.

Die Kombination aus deterministischer Simulationslogik, automatisierten Tests und Interaktionsprüfung schafft die Voraussetzungen für eine Anwendung, die gleichzeitig wissenschaftlich nachvollziehbar, technisch robust und im Lernprozess verständlich ist.

In der Architektur des Systems ist das Szenario-Management als Teil der Anwendungslogik umgesetzt und greift auf dieselben Normalisierungs- und Validierungsmechanismen zurück wie die Erstellung neuer Szenarien. Dadurch wird sichergestellt, dass importierte Daten denselben strukturellen Regeln unterliegen wie direkt im Generator erzeugte Daten. Die so realisierte Zustandsintegrität ist für die fachliche Nachvollziehbarkeit und die Vergleichbarkeit von Läufen von zentraler Bedeutung, da sie verhindert, dass visuelle Darstellung und Simulationskontext auseinanderlaufen.

6.6 Besondere technische Herausforderungen

Eine zentrale Herausforderung bestand in der Synchronisation von Simulationsgenauigkeit und interaktiver Bedienung. Einerseits musste die fachliche Korrektheit bei Präemption, Kontextwechsel und MLFQ-Regelwerk sichergestellt werden, andererseits sollte die Anwendung unmittelbar auf Eingaben reagieren.

Ein zweiter Schwerpunkt war die Sicherung von Reproduzierbarkeit. Dazu wurden deterministische Ausführungspfade und testbare Schnittstellen priorisiert. Automatisierte Tests prüfen sowohl Kernlogik als auch zentrale Integrationsfälle, wodurch Regressionen früh sichtbar werden [8].

Schliesslich ergab sich ein Trade-off zwischen Detailtiefe und Ressourcenverbrauch: Dichte Snapshot-Folgen verbessern Analyse und Wiedergabe, erhöhen jedoch Speicherbedarf. Diese Spannung wurde durch parametrisierbare Snapshot-Intervalle adressiert, sodass zwischen Genauigkeit und Laufzeitverhalten situationsgerecht balanciert werden kann.

6.7 Teststrategie und Qualitätssicherung

Die Qualitätssicherung konzentriert sich auf den Simulationskern und die Anwendungslogik. Die Tests werden mit Vitest ausgeführt und prüfen sowohl isolierte Simulationseigenschaften als auch zentrale Integrationspfade der Szenarioverwaltung [8]. Ein Schwerpunkt liegt auf der Deterministik: Wird ein identisches Szenario zweimal ausgeführt, müssen Gesamtdauer, Prozessabschluss, Ereignisfolge und Ausführungssegmente übereinstimmen.

Darüber hinaus werden die fachlich relevanten Verhaltensweisen der Verfahren mit kleinen, gezielt konstruierten Prozessmengen geprüft. Dazu gehören der Quantum-Ablauf und die Präemption bei Round Robin, die Präemption bei einer neuen Ankunft im präemptiven LCFS-Verfahren sowie die Herabstufung eines Prozesses nach Ablauf seines Quantums in der MLFQ. Ergänzend prüft die Eingabevalidierung, ob ungültige oder unvollständige

Szenariodaten normalisiert werden und ob die erzeugten Zeitsegmente gültige Grenzen besitzen.

Die Tests der Anwendungslogik ergänzen diese Kernprüfungen um die Verwaltung von Szenarien und Runs. Dabei wird unter anderem kontrolliert, ob aus einem Szenario und einem ausgewählten Run ein vollständiges Simulationsszenario erzeugt wird, ob Prozessdaten dabei kopiert werden und ob Änderungen an Szenario, Parametern oder aktivem Run zu einer veränderten Darstellung führen.

Zum Zeitpunkt der Erstellung dieser Arbeit umfasst die automatisierte Testsuite elf Testdateien mit insgesamt 34 erfolgreichen Tests. Die Tests sichern die technische Konsistenz der Implementierung ab, ersetzen jedoch keine fachliche Prüfung aller denkbaren Prozesskombinationen und keine empirische Untersuchung der Nutzung oder Lernwirkung. Die in Kapitel 7 dargestellten Produktbeispiele und Vergleichsstände dienen daher zusätzlich als nachvollziehbare Prüf- und Vergleichsfälle für die fachliche Aussagekraft der Anwendung.

7 Ergebnis und Produktpräsentation

Dieses Kapitel stellt das entwickelte Endprodukt vor und dokumentiert die zentralen Funktionen der Anwendung in ihrer fertigen Form. Im Unterschied zu einer klassischen empirischen Evaluation steht hier weniger die statistische Messung im Mittelpunkt, sondern die fachlich belastbare und visuell nachvollziehbare Produktwirkung. Ziel ist es, das entstandene Werkzeug als nutzbares Ergebnis eines engineering-orientierten Entwicklungsprozesses zu präsentieren und damit die praktische Umsetzung der fachlichen Anforderungen sichtbar zu machen.

7.1 Entwickeltes Endprodukt

Die entwickelte Webanwendung stellt einen interaktiven Scheduling-Visualizer dar, der verschiedene präemptive CPU-Scheduling-Verfahren in einem gemeinsamen Lern- und Analysekontext verknüpft. Die Anwendung erlaubt es, Tasks mit Ankunftszeit, Burst-Zeit, Priorität und weiteren Attributen ein Szenario zu definieren, einen Algorithmus auszuwählen und den Ablauf zeitlich sichtbar zu machen.

Das fertige Produkt verbindet damit drei zentrale Aspekte:

- technische Korrektheit der Scheduling-Logik,
- verständliche Darstellung des zeitlichen Ablaufs,
- und eine nutzerorientierte Oberfläche für Lehr- und Vergleichszwecke.

Damit erfüllt die Anwendung nicht nur die fachlichen Vorgaben eines Betriebssystem-Kapitels, sondern stellt zugleich ein tragfähiges und erweitertes Produkt dar, das im Studienkontext als didaktisches Werkzeug eingesetzt werden kann.

7.2 Produktüberblick und Nutzerfluss

Der typische Nutzerfluss beginnt mit dem Definieren eines Szenarios. Auf der Startseite sind die Systemgrundlagen, die Eingabeform und die Auswahl relevanter Task-Parameter direkt sichtbar. Nach dem Laden eines Szenarios kann der Nutzer den gewünschten Algorithmus auswählen und den Ablauf in verschiedenen Modi starten. Anschließend stehen eine Fokusansicht und eine Vergleichsübersicht zur Verfügung.

Die Anwendung wurde bewusst so aufgebaut, dass die Interaktion in logischer Reihenfolge abläuft: Eingabe, Auswahl, Ablaufsteuerung, Analyse und Vergleich. Dadurch bleibt der Nutzer bei jedem Schritt im Kontext des jeweiligen Lernziels und muss keine unnötigen Bedienhürden überwinden.

7.3 Produktpräsentation mit Screenshots

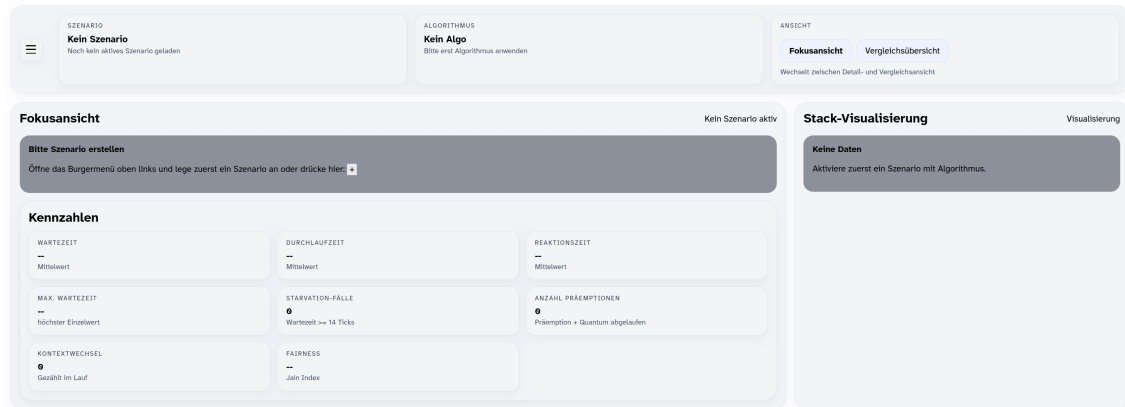


Figure 7.1: Startansicht der Anwendung mit leerem Zustand und erklärender Ausgangslage. Der Nutzer startet mit der Definition eines Szenarios und der Auswahl eines Scheduling-Verfahrens.

7.4 Umgesetzte Funktionalität

Die fertige Anwendung erfüllt die wesentlichen Anforderungen, die im Verlauf der Arbeit aus fachlicher, didaktischer und HCI-orientierter Sicht definiert wurden. Zu den wichtigsten Funktionen gehören:

- Erstellung und Bearbeitung von Task-Daten mit Ankunftszeit, Burst-Zeit und Priorität,
- interaktive Szenariogenerierung mit vordefinierten Basis-Szenarien und direkter Vorschau,
- Auswahl und Vergleich verschiedener Scheduling-Verfahren über einen klaren Algorithmusdialog,
- kontrollierte Wiedergabe mit Start, Pause, Schrittmodus und Zeitsprung,
- visuelle Darstellung als Zeitachsenmodell in Gantt-Form,

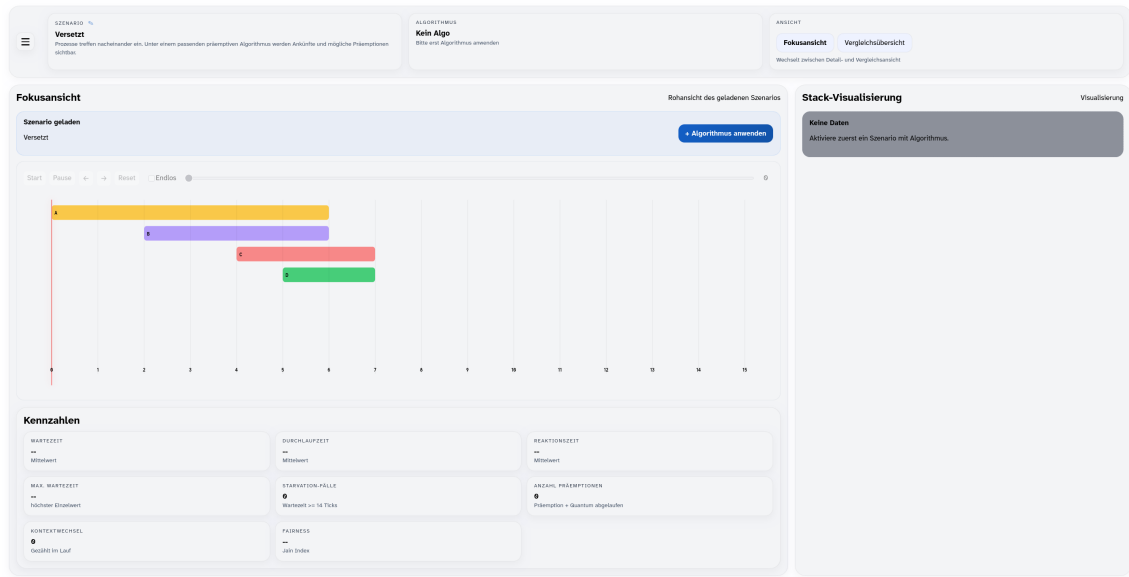


Figure 7.2: Ladezustand eines konfigurierten Szenarios. Die Task-Daten sind sichtbar, und die Simulation kann mit einer klaren Ablaufsteuerung initialisiert werden.

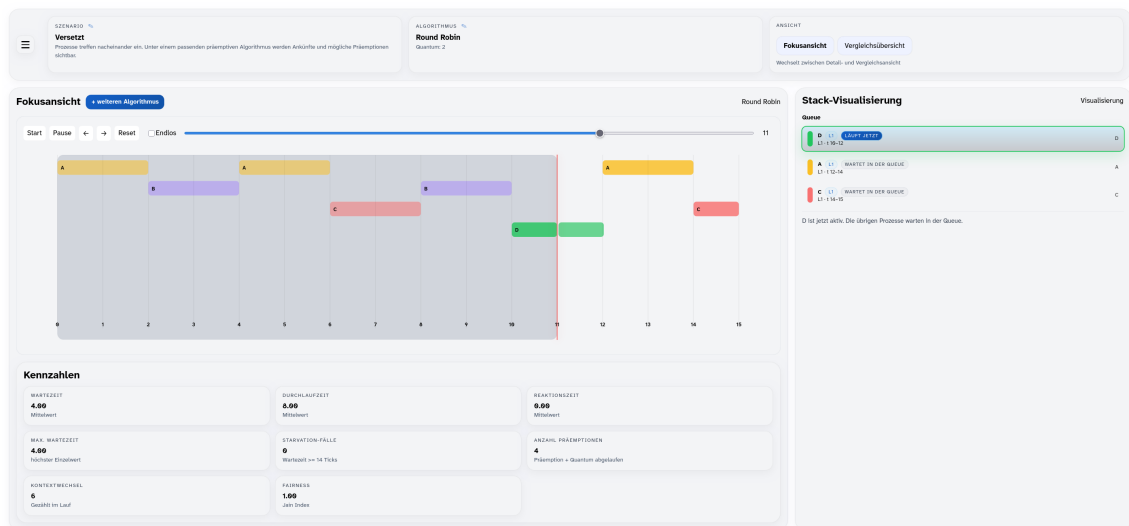


Figure 7.3: Fokusansicht der Simulation mit Gantt-Zeitachse, aktiver Task-Darstellung und Kennzahlbereich. Diese Ansicht zeigt den Ablauf des gewählten Scheduling-Verfahrens in einer verständlichen, zeitlich strukturierten Form.

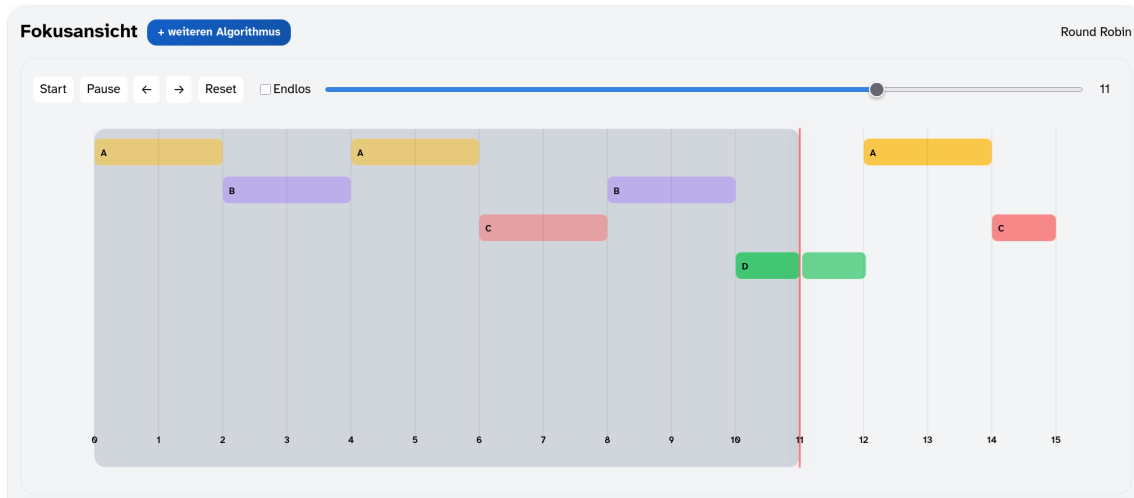


Figure 7.4: Detailansicht der Gantt-Darstellung. Einzelne Ausführungssegmente, Präemptionen und Task-Wechsel werden als konkrete Zeitabschnitte sichtbar gemacht.

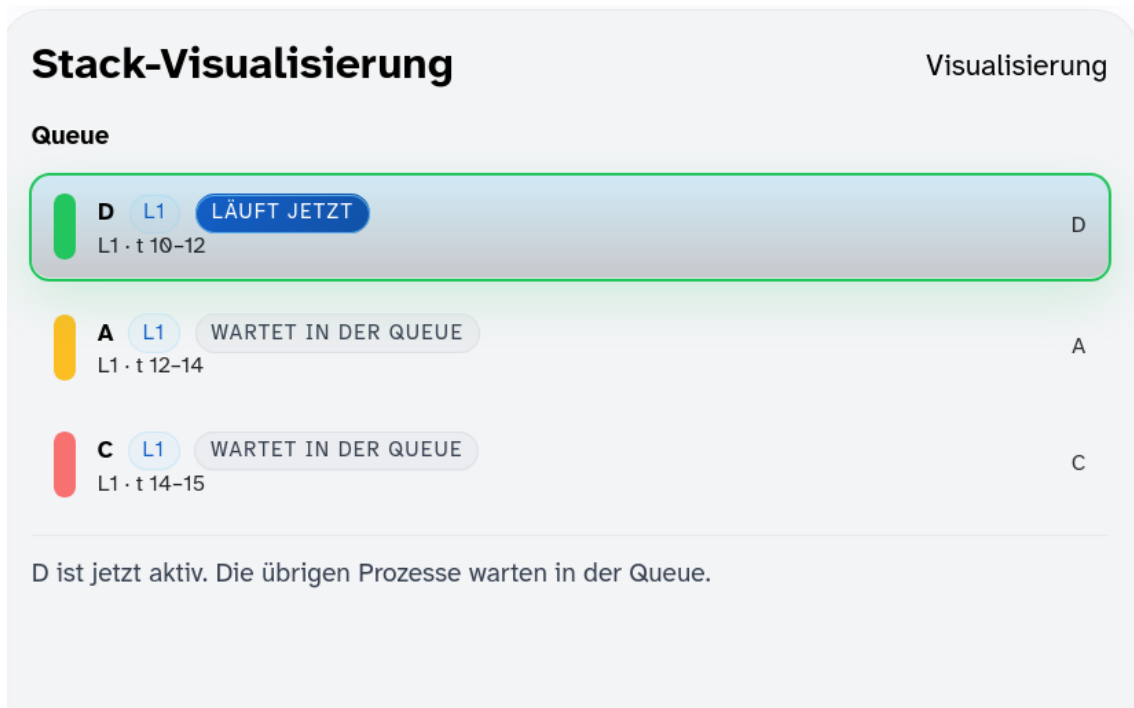


Figure 7.5: Darstellung des Task-Status in der Warteschlangenansicht. Der aktuelle Zustand und die Warteliste werden gleichzeitig präsentiert, wodurch die Auswahlentscheidung des Algorithmus nachvollziehbar wird.

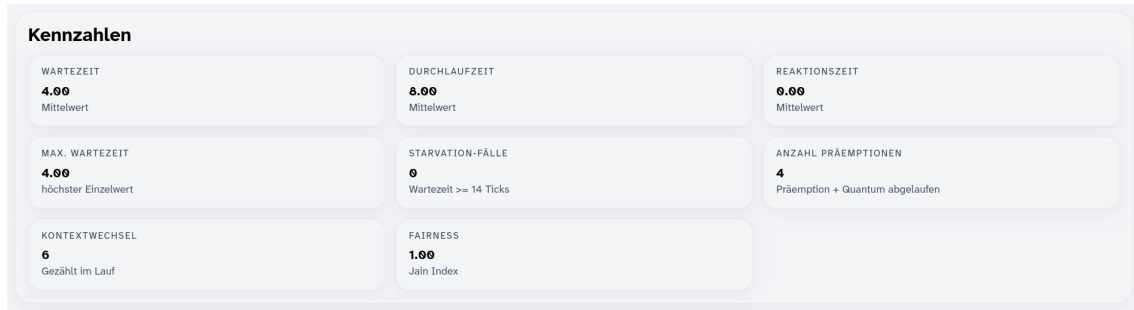


Figure 7.6: Kennzahlenbereich mit Warte-, Durchlauf- und Reaktionszeiten. Die Metriken geben eine kompakte fachliche Bewertung des Simulationslaufs wieder.

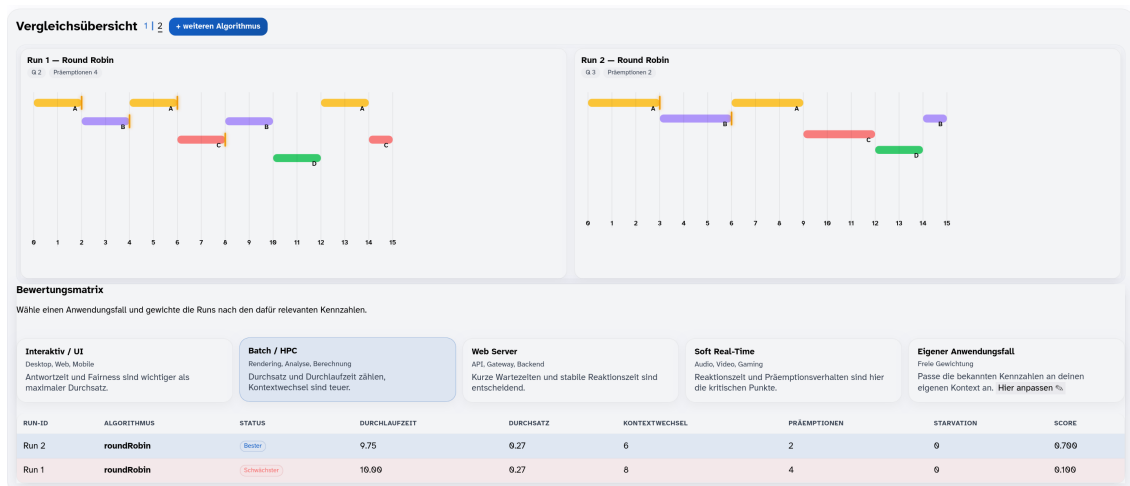


Figure 7.7: Vergleichsübersicht mit mehreren Läufen unter identischem Szenario. Die Darstellung ermöglicht den praktischen Vergleich verschiedener Scheduling-Verfahren anhand des selben Dateninputs.

- Ausgabe relevanter Kennzahlen pro Ablauf,
- Vergleich mehrerer Läufe auf identischem Szenario mit Bewertungs- und Matrixansicht.

Diese Funktionen zeigen, dass die Anwendung als echtes Produkt und nicht nur als akademische Demonstration umgesetzt wurde. Sie ermöglicht fachliche Analyse, didaktische Nachvollziehbarkeit und konkrete Produktnutzung in einem integrativen Ablauf, wobei die Benutzerführung durch Szenario-Generator, Vergleichsansichten und strukturierte Bedienpfade zusätzlich gestärkt wird.

7.5 Ergebnisbetrachtung

Das endständige Produkt erfüllt die ursprüngliche Zielsetzung der Arbeit: Es macht die Dynamik präemptiver Scheduling-Verfahren anschaulich, verständlich und vergleichbar. Die wichtigsten fachlichen Effekte, etwa Präemption, Kontextwechsel, Prioritätsverzug und Wartezeiten, werden nicht nur als Zahlenwerte gezeigt, sondern auch im zeitlichen Ablauf sichtbar gemacht.

Diese Kombination aus technischer Korrektheit und visueller Transparenz ist für das Produkt besonders wertvoll. Der Nutzer sieht nicht nur das Ergebnis eines Algorithmus, sondern auch die Entscheidungspfade, die zu diesem Ergebnis geführt haben. Dadurch wird die Anwendung zu einem nutzbaren Werkzeug für Lehre, Demonstration und fachliche Erörterung.

Das Produkt zeigt außerdem, dass die Arbeit eine konkrete technische Leistung erbracht hat: Es wurde eine vollständig nutzbare, interaktive Anwendung entwickelt, die fachlich korrekte Simulation, verständliche Visualisierung und eine klare, strukturiert gestaltete Benutzerführung verbindet. Gerade diese Kombination ist im Studiengang als engineering-orientierte Leistung besonders aussagekräftig, weil sie den Übergang von abstrakter Theorie zu konkreter Softwarelösung sichtbar macht.

Die Produktpräsentation zeigt zugleich, dass die Bachelorarbeit nicht nur ein theoretisches Dokument, sondern ein konkret umgesetztes Softwaresystem ist. Diese Perspektive passt zur engineering-orientierten Ausrichtung der Arbeit und entspricht zugleich dem fachlichen Anspruch des Studiengangs, in dem methodische Genauigkeit, technische Umsetzung und verständlicher Produktaufbau gleichwertig berücksichtigt werden müssen.

8 Fazit und Ausblick

Die vorliegende Arbeit zeigt, dass eine didaktisch wirksame und fachlich belastbare Visualisierung präemptiver CPU-Scheduling-Verfahren als interaktive Webanwendung realisiert werden kann. Die zentrale Leistung besteht nicht nur in der technischen Umsetzung, sondern in der Verbindung von Simulationslogik, erklärender Visualisierung und lernorientierter Bedienung. Dadurch wird die im Betriebssystemkontext zentrale Idee der Prozessverwaltung für Lernende nicht nur theoretisch beschrieben, sondern unmittelbar erfahrbar gemacht.

Die entwickelte Anwendung erfüllt die ursprüngliche Zielsetzung: Sie macht Scheduling-Entscheidungen sichtbar, verknüpft diese mit relevanten Kennzahlen und erlaubt eine nachvollziehbare Gegenüberstellung verschiedener Verfahren auf identischem Szenario. In diesem Sinne ist das Ergebnis ein produktiver Beitrag zur didaktischen Vermittlung von Betriebssystemprinzipien. Die Kombination aus Gantt-Darstellung, Queue-Zustand, Kennzahlen und Vergleichsansicht ist besonders geeignet, um die Dynamik präemptiver Verfahren zu verstehen, ohne die fachliche Genauigkeit zu verlieren.

8.1 Ergebnis der Arbeit

Die Arbeit verbindet theoretische Grundlagen, fachliche Anforderungen, HCI-Überlegungen und technische Umsetzung in einem konsistenten Entwurf. Die Simulation basiert auf einem gemeinsamen, deterministischen Modell, das verschiedene Scheduling-Strategien vergleichbar macht. Gleichzeitig werden die Ergebnisse in einer gestalteten Oberfläche präsentiert, die den Lernprozess durch direkte Visualisierung und nachvollziehbare Zustandswechsel unterstützt.

Das Produkt ist damit nicht bloß eine Demonstrationsumgebung, sondern ein nutzbares Werkzeug für Lehre, Analyse und fachliche Diskussion. Gerade weil die Anwendung unterschiedliche Algorithmen mit derselben Datenbasis vergleicht, wird klar erkennbar, wie sich Entscheidungen in der Auswahlstrategie auf Wartezeit, Durchlaufzeit, Reaktionszeit und Kontextwechsel auswirken.

8.2 Ausblick

Für eine mögliche Weiterentwicklung bietet sich eine Erweiterung in mehreren Richtungen an. Eine erste Erweiterung wäre die Ergänzung weiterer Scheduling-Verfahren oder die Einbindung realistischerer Lastmodelle. Ebenfalls sinnvoll ist die Verfeinerung der Bewertungslogik, etwa durch zusätzliche Gewichtungsoptionen für didaktische oder fachliche Lernziele. Auch eine detailliertere Export- und Analysefunktion könnte die fachliche Nachnutzung der erzeugten Szenarien verbessern.

Ein weiterer sinnvoller Schritt wäre die Überführung der Anwendung in einen stärker didaktisch strukturierten Verlauf, etwa mit erklärenden Lehrpfaden, Aufgabenstellungen oder vorgefertigten Vergleichsszenarien. Dadurch könnte die Anwendung nicht nur als Visualisierungswerkzeug, sondern auch als Lernumgebung mit klarer didaktischer Begleitung fungieren.

Die bestehende Umsetzung bildet daher eine solide Grundlage für weiterführende Arbeiten im Bereich interaktiver Betriebssystemvisualisierung. Sie zeigt, dass technische Korrektheit, Nutzbarkeit und didaktische Klarheit in einem gemeinsamen Produkt zusammengeführt werden können.

Bibliography

- [1] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, 13(3):259–290, 2002. Definitive meta-study: Wichtig fuer Argumente zur Lehrwirksamkeit von Visualisierungen.
- [2] L. W. C. Meing and D. R. Tandingan. Exploring multilevel feedback queue combinations and regression-based time quanta in scheduling algorithms. *Journal of Information Systems Engineering and Management (JISEM)*, 2025. Studie zu MLFQ-Kombinationen und dynamischer Zeitscheibenwahl.
- [3] T. L. Naps and Others. Exploring the role of visualization and engagement in computer science education. In *SIGCSE Technical Symposium on Computer Science Education (proceedings)*, 2002. Grundlagen zur Engagement-Taxonomie (viewing vs. interacting vs. constructing).
- [4] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 4 edition, 2016. Formale Algorithmenbetrachtung und Performance-Metriken.
- [5] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10 edition, 2018. Standardwerk: Scheduling-Definitionen und Metriken.
- [6] John T. Stasko, John Domingue, Marc H. Brown, and Bruce A. Price, editors. *Software Visualization: Programming as a Multimedia Experience*. MIT Press, 1998. Designprinzipien und Taxonomien fuer Software/Algorithmus-Visualisierung.
- [7] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2014. Konzeptuelle Tiefe zu Praelemption, Kontextwechseln und Scheduler-Design.
- [8] Vitest Team. Vitest - unit testing for vite. Test-Runner im Projekt.

9 Anhang

Der Anhang enthält ergänzende Materialien, zum Beispiel zusätzliche Screenshots, Testfälle und tabellarische Auswertungen.

Die Zustandsübergänge der Benutzeroberfläche vom initialen Leerzustand bis zum geladenen Szenario sind in Kapitel 6 dokumentiert. Die zugehörigen Originaldateien liegen im Bildverzeichnis des Projekts und können bei Bedarf für eine umfangreichere Anhangsdokumentation verwendet werden.